



Assignment 2

Computernetze und ihre Leistung

von:

Michael Heise

Linus Henn

Luca Lars Stoever

Matrikelnr.: 537385

Matrikelnr.: 576038

Matrikelnr.: XXXXX

Kurs:

Computernetze und ihre Leistung SoSe 2026

Dozent: **Ralph-Günther Holz**

Münster, 28. Juli 2026

Contents / Inhaltsverzeichnis

1	Aufbau	1
1.0.1	Link Einstellungen	1
1.0.2	TCP Staukontrolle	1
1.0.3	iperf3	2
1.0.4	udp	2
1.0.5	tcp	2
1.0.6	PCAP Capture	2
1.0.7	Shell	3
2	Methodik	3
3	UDP	3
3.1	Durchführung	3
3.2	Ergebnisse	4
3.3	Diskussion	5
4	TCP	6
4.1	Parameter und Durchführung	6
4.2	Ergänzende Methodik	6
4.3	Ergebnisse	6
4.4	Diskussion	6
5	TCP unter Jitter	6
5.1	Parameter und Durchführung	6
5.2	Ergänzende Methodik	6
5.3	Ergebnisse	6
5.4	Diskussion	6
6	Vergleich von TCP Reno und TCP Cubic	6
6.1	Parameter und Durchführung	6
6.1.1	Setup	6
6.1.2	Durchführung mit TCP Reno	7
6.1.3	Durchführung mit TCP Cubic	7
6.1.4	Messdaten	7
6.2	Ergänzende Methodik	7
6.2.1	Durchsatzanalyse	7
6.2.2	Analyse des Staukontrollfensters	7
6.2.3	Retransmissionen und Paketverluste	7

6.2.4	Auswertungsskripte	7
6.3	Ergebnisse	8
6.3.1	Durchsatz	8
6.3.2	Entwicklung des Staukontrollfensters	8
6.3.3	Retransmissionen, Paketverluste und deren Einfluss	10
6.3.4	Zusammenfassender Vergleich	11
6.4	Diskussion	11
6.4.1	Wachstum und Reaktion	11
6.4.2	Performance, Robustheit und Stabilität	12
6.4.3	Praktischer Einsatz	12
6.4.4	Grenzen des Labs	13

1 Aufbau

Als Grundlage der hier durchgeführten Experimente dient ein virtuelles Netzwerk ausgeführt in Docker. Dabei werden zwei Hosts (n1, n2) miteinander verknüpft. Es ist möglich, TCP und UDP Verbindungen aufzubauen und die übertragenen Pakete in eine PCAP Datei zu speichern. Zudem können einige Parameter für den Link zwischen den Hosts eingestellt werden.

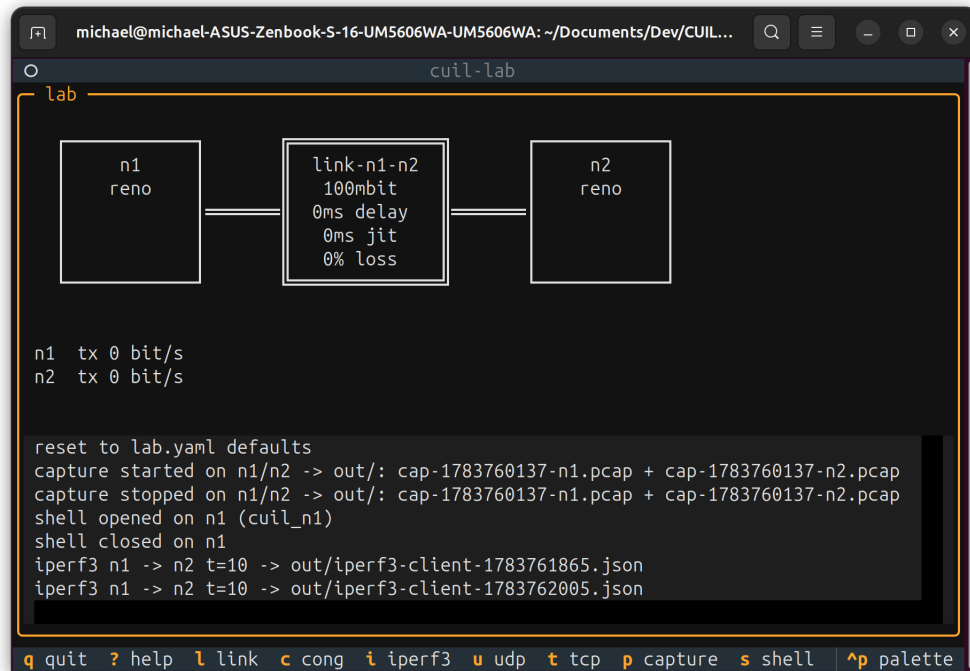


Abbildung 1: Docker Netzwerk

1.0.1 Link Einstellungen

Der Link zwischen den Hosts kann in der Bandbreite, Latenz, Verzögerung und Paketverlust konfiguriert werden.

Standardmäßig ist der Link mit folgenden Parametern konfiguriert:

- Bandwidth: 100 Mbit/s
- Delay: 0 ms
- Jitter: 0 ms
- Paketverlust: 0 %
- Burst: 32kbit

1.0.2 TCP Staukontrolle

Für jeden Host kann ein Standard für die Staukontrolle gewählt werden. Zur Auswahl stehen TCP Reno und TCP CUBIC. Uns ist nicht bekannt, welcher Standard genutzt wird, da diese kontinuierlich weiterentwickelt werden. Folgende Quellen geben einen Überblick über die beiden Staukontrollen, auch wenn sich die genaue Version des Standards unterscheiden kann.

Standardmäßig ist die Staukontrolle auf TCP Reno eingestellt.

- TCP Reno: [2]

- TCP CUBIC: [5]

1.0.3 iperf3

iperf3 ist ein Tool, um Netzwerkleistung in IP Netzwerken zu messen. Das Programm misst den Durchsatz, Paketverlust und andere Parameter [1]. In der Docker Umgebung muss ein Host als Server und ein Host als Client angegeben werden anschließend noch eine Messdauer und ein Messintervall. Eine Messung erzeugt eine JSON und eine Log-Datei, die die Ergebnisse der Messung enthalten. In der Log-Datei ist zu sehen, dass für jedes Intervall gemessen wird wie viele Daten übertragen wurden, daraus ergibt sich der gemessene Durchsatz und die Übertragungsrate für den Link.

1.0.4 udp

Das UDP Tool in der Docker Umgebung kann genutzt werden, um UDP Verbindungen zwischen den Hosts aufzubauen. Dabei müssen Server und Client angegeben werden, sowie die Anzahl der Pakete, das Intervall und die Paketgröße. Beim Ausführen sendet der Client UDP Pakete an den Server. Für den ganzen Vorgang wird eine Log-Datei pro Endpunkt erstellt, die die Anzahl der gesendeten und empfangenen Pakete enthält.

Die Standardeinstellungen für das UDP Tool sind:

- Server: n2
- Client: n1
- Count: 10
- Interval: 0.1 s
- size: 64 B

1.0.5 tcp

Das TCP Tool in der Docker Umgebung kann genutzt werden, um TCP Verbindungen zwischen den Hosts aufzubauen. Dabei müssen Server und Client angegeben werden, sowie die Anzahl der Pakete, das Intervall und die Paketgröße. Zusätzlich zu diesen Einstellungen, die wir auch für UDP nutzen, kann eingestellt werden, ob eine persistente Verbindung genutzt werden soll, oder ob für jedes Paket eine neue Verbindung aufgebaut werden soll.

Die Standardeinstellungen für das TCP Tool sind:

- Server: n2
- Client: n1
- Count: 10
- Interval: 0.1 s
- size: 64 B
- Mode: persistent

1.0.6 PCAP Capture

In der Docker Umgebung ist es möglich, die übertragenen Pakete in eine PCAP Datei zu speichern. Dazu kann ein Capture mit p gestartet werden. Anschließend wird pro Host eine PCAP Datei erstellt, die alle Pakete enthält, die über den Link übertragen wurden bis das Capture wieder gestoppt wird.

1.0.7 Shell

In der Docker Umgebung ist es möglich, eine Shell auf den Hosts zu öffnen. Mit der Shell können Hosts konfiguriert und Befehle ausgeführt werden, die nicht über die Docker Umgebung möglich sind.

2 Methodik

Für die Experimente wurde das im Aufbau 1 beschriebene virtuelle Netzwerk genutzt. Dabei wurde für jedes Experiment eine Datei erzeugt, die den Datenverkehr im Netzwerk aufzeichnet. iperf3 1.0.3 erzeugt JASON und TXT-Dateien, die direkt die Ergebnisse der Messung enthalten. Teilweise werden auch Shell Befehle 1.0.7 genutzt, um die Experimente zu starten und zu stoppen oder um Ausgaben der Experimente zu erhalten. PCAP Dateien werden von dem 1.0.6 beschriebenen Capture Tool erzeugt und können anschließend mit Wireshark [4] analysiert werden hinsichtlich bestimmter Fragestellungen untersucht werden. Wireshark ist ein Tool, um Netzwerkverkehr zu analysieren und kann PCAP Dateien öffnen und die enthaltenen Pakete darstellen. Einzelne Pakete können byteweise analysiert werden. Wireshark erlaubt einzelne Parameter und Flags der Verbindungen zu analysieren und die Pakete nach bestimmten Kriterien zu filtern.

3 UDP

Das UDP (User Datagram Protocol) ist ein verbindungsloses Protokoll der Transportschicht. In diesem Versuch soll der verlustfreie und verlustbehaftete UDP Datenverkehr zwischen zwei Hosts untersucht werden.

3.1 Durchführung

Für die Durchführung wurde das Docker Netzwerk 1 benutzt. Es wurde zweimal der UDP Datenverkehr zwischen zwei Hosts untersucht. Die Docker Umgebung wurde bei der Standardkonfiguration belassen. Einzig die Link-Parameter 1.0.1 wurden beim zweiten Durchlauf angepasst, um den Paketverlust zu simulieren:

- Bandwidth: 100 Mbit/s
- Delay: 0 ms
- Jitter: 0 ms
- Paketverlust: 30 %
- Burst: 32kbit

In beiden Fällen wurden folgende Schritte durchgeführt:

1. Starten der Docker Umgebung
2. Einstellen der Link Eigenschaften 1.0.1
3. Starten des Netzwerkverkehr-Mitschnitts 1.0.6
4. Starten der UDP Verbindung zwischen den Hosts n1 (UDP Client) und n2 (UDP Server) 1.0.4
 - Server: n2
 - Client: n1
 - Count: 30
 - Interval: 1 s

- size: 64 B
5. Beenden des Netzwerkverkehr-Mitschnitts 1.0.6
 6. Schließen der Docker Umgebung

Nachdem die Log- und PCAP-Dateien erstellt wurden, wurden diese ausgewertet und die Ergebnisse dokumentiert.

3.2 Ergebnisse

Die Log- und PCAP-Dateien treffen dieselben Aussagen über den UDP Datenverkehr zwischen den Hosts. Da die Log-Dateien weniger unnötige Informationen enthalten, werden diese für die Auswertung genutzt. Die PCAP-Dateien enthalten neben den UDP Paketen auch ARP Anfragen und Antworten, die für die Auswertung nicht relevant sind. Bei dem Link ohne Paketverlust wurden alle 30 Pakete erfolgreich mit nahezu keiner Verzögerung übertragen. Bei dem Link mit Paketverlust kamen nur 22 der 30 Pakete beim Server an. Das ist näherungsweise konsistent mit dem eingestellten Paketverlust von 30 % (8 von 30 Paketen, also ca. 27 %). Die verlorenen Pakete wurden erkannt, da sie vom Client gesendet wurden, aber nicht in der Log-Datei des Servers auftauchten.

Paket Nummer	Gesendet durch Client	Empfangen durch Server
1	J	J
2	J	J
3	J	N
4	J	J
5	J	J
6	J	J
7	J	J
8	J	J
9	J	N
10	J	J
11	J	J
12	J	N
13	J	N
14	J	N
15	J	N
16	J	J
17	J	J
18	J	J
19	J	N
20	J	J
21	J	N
22	J	J
23	J	J
24	J	J
25	J	J
26	J	J
27	J	J
28	J	J
29	J	J
30	J	J

Tabelle 1: Übertragungsstatus der Pakete zwischen Server und Client mit 30 % Paketverlust

Den Logs lässt sich außerdem entnehmen, dass in jedem Fall genau 30 Pakete vom Client gesendet wurden. Die verlorenen Pakete wurden nicht erneut gesendet, da UDP ein verbindungsloses Protokoll ist und keine Mechanismen zur Fehlerbehandlung implementiert sind.

3.3 Diskussion

Als Protokoll der Transportschicht wird der UDP Payload an die Anwendungsschicht weitergeleitet. Geht ein Paket verloren, so wird die Anwendungsschicht nicht darüber informiert. Die Anwendungsschicht kann also durch verlustbehafteten UDP Datenverkehr beeinflusst werden und muss gegebenenfalls selbst Mechanismen implementieren, um mit Paketverlusten umzugehen.

Auswirkung auf Anwendungsschicht

Die Anwendungsschicht erhält durch das UDP Protokoll lückenhafte Daten. Auswirkungen können unkritisch sein. Bei Video Streaming oder Voice over IP (VoIP) würde die Stimme oder das Bild kurzzeitig ausfallen oder verzerrt werden, aber die Anwendung würde weiterlaufen. Online Spiele, die auf UDP basieren, würden ebenfalls kurzzeitig einfrieren oder verzerrt dargestellt werden, aber die Anwendung würde weiterlaufen. Außerdem wäre ein erneutes Senden nutzlos, da die verlorenen Pakete nicht mehr relevant sind. Webseiten oder Dateiübertragungen, die über UDP mit Verlust geladen werden, wären unvollständig, und funktional eingeschränkt. Die Anwendung würde nicht mehr korrekt funktionieren, da die verlorenen Pakete erneut gesendet werden müssten, um die Funktionalität der Anwendung zu gewährleisten. Die Anwendungsschichtprotokolle können unterschiedlich auf Paketverlust reagieren.

Umgang mit Paketverlust

Bei Anwendungen bei denen alte Pakete nicht mehr verwendet werden können, oder ein Qualitätsverlust in Kauf werden kann, wird der Paketverlust toleriert.

Bei Anwendungen bei denen die verlorenen Pakete erneut gesendet werden müssen, um die Funktionalität der Anwendung zu gewährleisten, muss der Paketverlust erkannt und die verlorenen Pakete erneut gesendet werden. Einige Protokolle wie Quick UDP Internet Connections (QUIC) implementieren Flusskontrolle, Staukontrolle, Datenintegrität, aber ebenfalls Retransmission von verlorenen Paketen auf der Anwendungsschicht. Es werden ähnliche Mechanismen wie bei TCP implementiert, um die verlorenen Pakete erneut zu senden. Eine Kombination von Sequenznummern und Bestätigungen, sowie Timeouts, werden genutzt, um verlorene Pakete zu erkennen und erneut zu senden.

4 TCP

4.1 Parameter und Durchführung

4.2 Ergänzende Methodik

4.3 Ergebnisse

4.4 Diskussion

5 TCP unter Jitter

5.1 Parameter und Durchführung

5.2 Ergänzende Methodik

5.3 Ergebnisse

5.4 Diskussion

6 Vergleich von TCP Reno und TCP Cubic

Hinweis zur Gliederung: Die Darstellung folgt dem Ablauf des Experiments und weicht deshalb von der Reihenfolge der Teilaufgaben a bis d ab. Die Teilaufgaben 3a und 3b werden in Abschnitt 6.1 behandelt. Die Analyse aus Teilaufgabe 3c wird in Abschnitt 6.2 beschrieben. Vergleich und Bewertung gemäß Teilaufgabe 3d erfolgen in den Abschnitten 6.3 und 6.4.

6.1 Parameter und Durchführung

6.1.1 Setup

Beide Experimente nutzen das gleiche Netzwerk (vgl. Abschnitt 1) mit identischen Linkparametern. Die Konfigurationen sind in `lab-reno.yaml` und `lab-cubic.yaml` definiert. Die gemeinsamen Linkparameter sind in Tabelle 2 zusammengefasst.

Parameter	Wert
Bandbreite	1 Mbit/s
Verzögerung (One-Way)	20 ms
Jitter	0 ms
Paketverlustrate	3 %
Burst	32 kbit

Tabelle 2: Linkparameter für beide Experimente

Zur Messung wird `iperf3` (vgl. Abschnitt 1.0.3) verwendet. Der Sender (n1), als `iperf3-Client`, sendet über 30 Sekunden Daten an den Empfänger (n2), der als `iperf3-Server` agiert. Die Ergebnisse werden im JSON Format auf Sekundenbasis als Messerintervalle gespeichert. Zudem werden PCAP Dateien auf beiden Seiten generiert.

6.1.2 Durchführung mit TCP Reno

Für den ersten Durchlauf wird TCP Reno als Staukontrolle konfiguriert (vgl. `lab-reno.yaml`). Anschließend wird Messung mittels `iperf3` gestartet und Traffic aufgezeichnet.

6.1.3 Durchführung mit TCP Cubic

Im zweiten Experiment wird die Staukontrolle auf TCP CUBIC gewechselt (vgl. `lab-cubic.yaml`). Die restlichen Einstellungen bleiben unverändert. Zuletzt wird unter gleichen Bedingungen die Messung durchgeführt.

6.1.4 Messdaten

Aus jedem Experiment ergeben sich drei Dateien:

- `iperf3_{reno,cubic}.json` : Durchsatz, Retransmissionen und `cwnd`
- `tcp_{reno,cubic}.pcap` : PCAP auf Senderseite
- `tcp_{reno,cubic}_receiver.pcap` : PCAP auf Empfängerseite

6.2 Ergänzende Methodik

6.2.1 Durchsatzanalyse

Der Durchsatz kann direkt aus den `iperf3` Dateien entnommen werden. Für jedes der Messintervalle wird Bitrate und Zeitstempel extrahiert.

6.2.2 Analyse des Staukontrollfensters

Zur Analyse der Staukontrollfenster wird `tcptrace` [3] auf die Sender PCAPs angewendet. Das Tool erzeugt `xpl`-Dateien mit dem Outstanding-Data-Verlauf, also der Menge an gesendeten, aber noch nicht bestätigten Daten. Laut Man-Page approximiert dieser Wert das Congestion Window (*estimated congestion window*) [3]. Aus der durch `tcptrace` erzeugten Datei werden die Daten der Kategorie `red` entnommen, die den Outstanding Wert gegenüber der Zeit darstellen.

6.2.3 Retransmissionen und Paketverluste

Retransmissionen werden mit `tshark` aus den Sender PCAPs extrahiert. Dabei wird durch `tcp.analysis.retransmission` alle Pakete gefiltert, die von TCP als Retransmission erkannt werden. Für jedes dieser Pakete wird der relative Zeitstempel gespeichert.

Zur Bestimmung der Paketverluste werden Sender und Empfänger PCAPs verglichen. Aus beiden Aufzeichnungen werden alle Pakete mit (`tcp.len > 0`) und (`ip.id`) extrahiert. Anschließend wird in Python die Mengendifferenz berechnet: $Lost = \{x \in \text{Sender} \mid x \notin \text{Receiver}\}$.

6.2.4 Auswertungsskripte

Die gesamte Analyse ist über den Orchestrator `analyze.sh` idempotent ausführbar. Dieses ruft nacheinander `tcptrace-util.sh` (in einem `debian:bookworm-slim` Docker Container), `tshark-util.sh` und `analyze.py` (via `uv run`) auf. Das Python Skript nutzt `parsers.py` für das Parsing der Dateien sowie `plots.py` für die Erzeugung der Diagramme mit `matplotlib`. Voraussetzungen für die Ausführung sind `docker`, `tshark` und `uv`.

6.3 Ergebnisse

6.3.1 Durchsatz

Abbildung 2 zeigt den Durchsatzverlauf beider Algorithmen über die eingestellte Dauer.

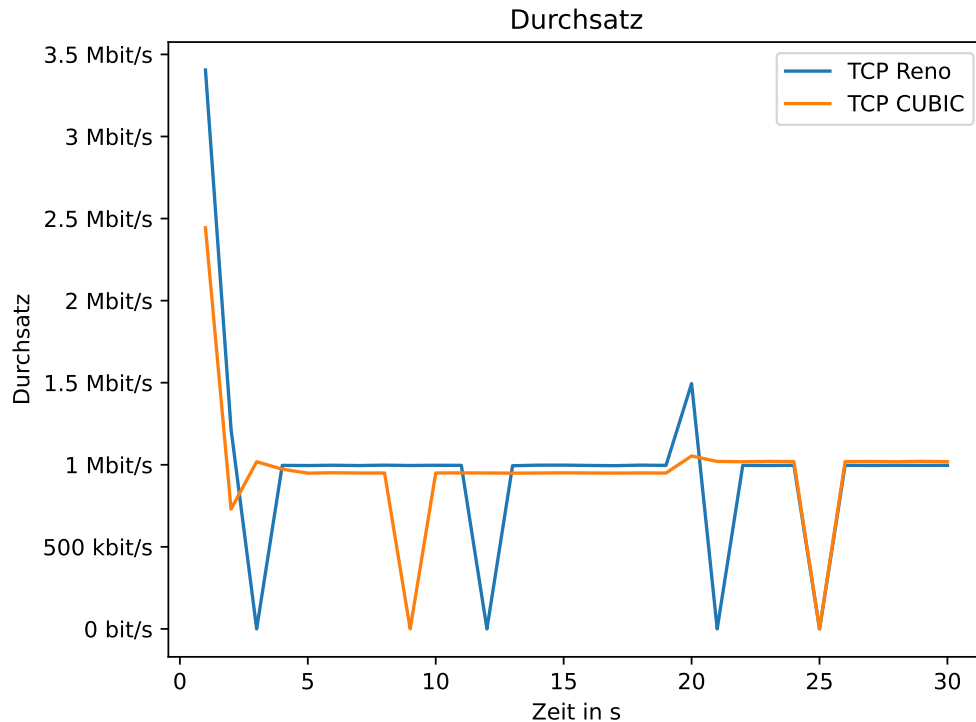


Abbildung 2: Durchsatzverlauf von TCP Reno und TCP CUBIC

Der mittlere Durchsatz wird von iperf3 als `bits_per_second` unter `end.sum_sent` in `iperf3_{reno,cubic}.json` dargestellt. TCP Reno misst ca. 968 kbit/s, TCP CUBIC ca. 956 kbit/s. Die Werte liegen damit nahe an der Konfiguration von 1 Mbit/s Durchsatz. Beide Algorithmen zeigen Schwankungen, welche auf die konfigurierte Paketverlustrate von 3 % zurückzuführen sind. Aus `end.sum_sent` ergeben sich `bytes` und `retransmits`: Reno überträgt 3.628.688 Bytes mit 80 Retransmissionen, CUBIC 3.586.696 Bytes mit 71 Retransmissionen.

6.3.2 Entwicklung des Staukontrollfensters

Abbildung 3 zeigt den Verlauf der Outstanding Data (approx. Congestion Window) beider Algorithmen im Vergleich.

Bei TCP Reno zeigt sich das typische Muster: `cwnd` wächst linear in der Congestion Avoidance, bis ein Paketverlust erkannt wird, woraufhin es halbiert wird. Aus der `tcptrace` Zusammenfassung (`reno-summary.txt`) ergeben sich `max owin` = 70.953 Bytes und `avg owin` = 12.232 Bytes. Zum Vergleich zeigt iperf3 `snd_cwnd` im Bereich von 2.896 bis 27.512 Bytes (Mittel: 10.667 Bytes). Abbildung 4 zeigt den Zusammenhang mit den Paketverlusten: 82 verlorene Pakete (Verlustrate 3,4 %).

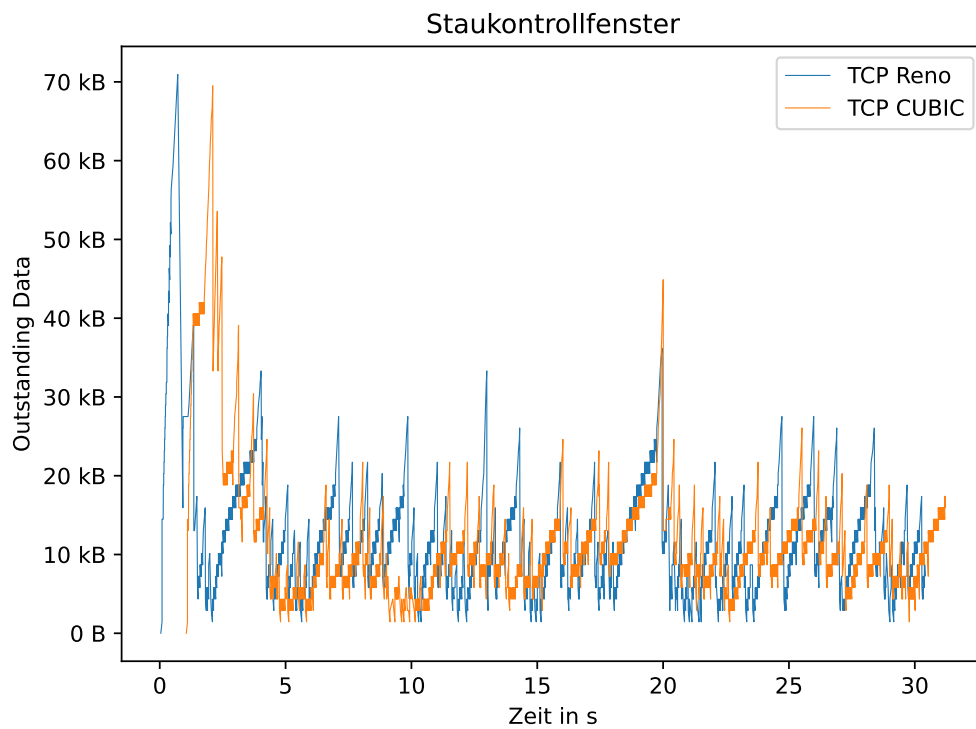


Abbildung 3: cwnd von TCP Reno und TCP CUBIC

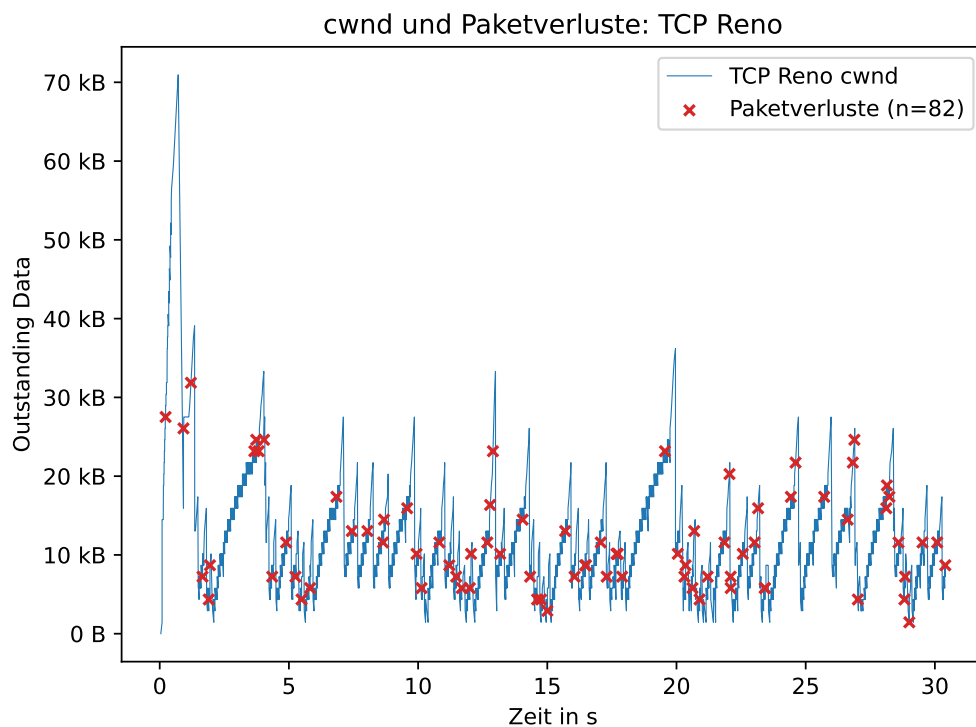


Abbildung 4: cwnd und Paketverluste von TCP Reno

TCP CUBIC zeigt ein aggressiveres Wachstumsverhalten. Im Verlauf zeigt sich nach jedem Verlustereignis ein Abfall des cwnd auf etwa 70 % des zuvor erreichten Maximums. Der Anstieg verläuft

zunächst steil, flacht beim Erreichen des vorherigen Maximalwerts sichtbar ab und geht dort in ein Plateau über. Laut `cubic-summary.txt`: `max owin = 69.505 Bytes`, `avg owin = 11.980 Bytes`. Vergleichsweise zeigt `iperf3 snd_cwnd` von 1.448 bis 28.960 Bytes (Mittel: 10.570 Bytes). Abbildung 5 zeigt die Paketverluste: TCP CUBIC weist 70 verlorene Pakete auf (Verlustrate 2,9 %).

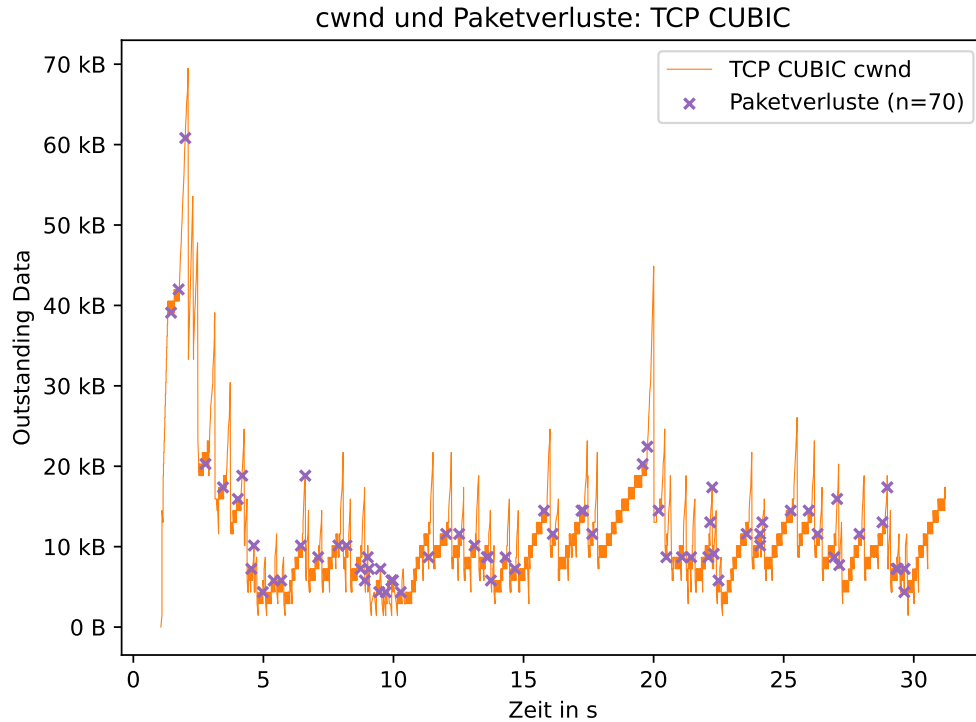


Abbildung 5: cwnd und Paketverluste von TCP CUBIC

6.3.3 Retransmissionen, Paketverluste und deren Einfluss

Abbildung 6 stellt die kumulierten Retransmissionen beider Algorithmen als Stufenfunktion dar.

TCP Reno zeigt insgesamt 81 Retransmissionen, TCP CUBIC 71. Die Retransmissionsrate steigt bei beiden Algorithmen annähernd gleichmäßig an, was mit der Paketverlustrate von 3 % konsistent ist.

Der Zusammenhang zwischen Paketverlusten und Staukontrollfenster ist wie bereits beschrieben in Abbildung 4 und Abbildung 5 sichtbar. Eine regressionsbasierte Analyse des Wachstumsverhaltens erfolgt in Abschnitt 6.4.

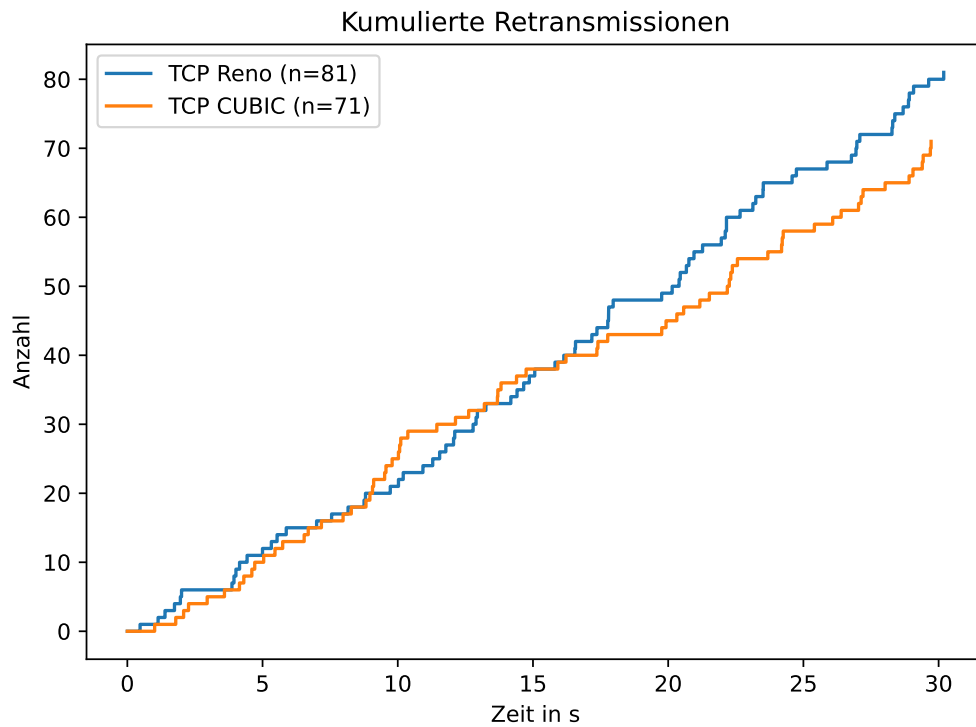


Abbildung 6: Kumulierte Retransmissionen von TCP Reno und TCP CUBIC

6.3.4 Zusammenfassender Vergleich

Tabelle 3 fasst die wichtigsten Kennwerte beider Algorithmen zusammen (Entnommen aus `iperf3-{reno,cubic}.json` und `{reno,cubic}-summary.txt` und `parse_packet_losses()` in `parsers.py`).

Kennzahl	TCP Reno	TCP CUBIC
Mittlerer Durchsatz	968 kbit/s	956 kbit/s
Übertragene Bytes	3.628.688	3.586.696
Retransmissionen	81	71
Paketverluste	82	70
Verlustrate	3,4 %	2,9 %
Max. Outstanding Data	≈ 71 kB	≈ 70 kB
Mittl. Outstanding Data	≈ 12 kB	≈ 12 kB
RTT (Mittel)	91,4 ms	86,2 ms
RTT (Min / Max)	40,3 / 437,2 ms	40,1 / 340,1 ms
Triple Dupacks	59	52

Tabelle 3: Vergleich der Kennwerte

6.4 Diskussion

6.4.1 Wachstum und Reaktion

Abbildung 7 zeigt die `cwnd` Verläufe beider Algorithmen mit Regressionskurven. Dabei wird für jede Congestion Avoidance Regim (Wachstumsphase zwischen zwei Verlustereignissen) ein Polynom des passenden Grades gefittet: linear (Grad 1) für Reno, kubisch (Grad 3) für CUBIC.

Für TCP Reno bestätigt der lineare Fit den AIMD Mechanismus mit einem mittleren $\bar{R}^2 = 0,942$

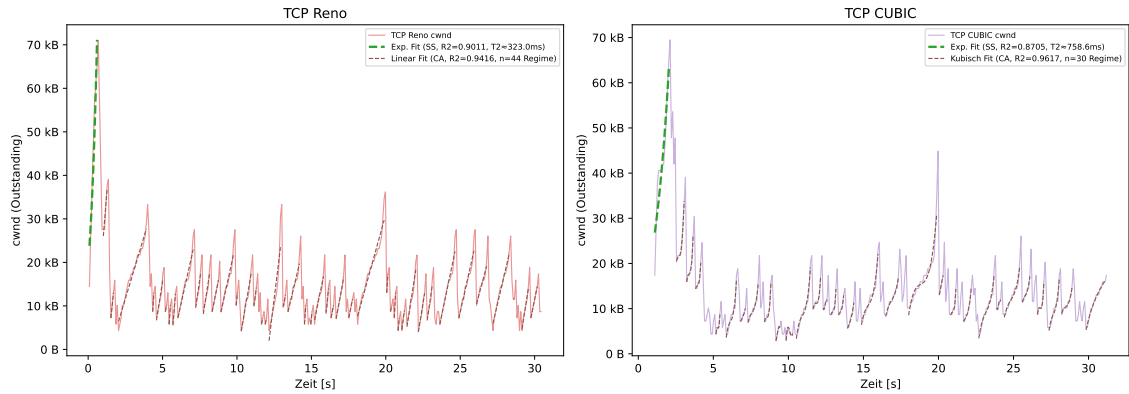


Abbildung 7: Regressionsbasierte Analyse: Congestion Avoidance Fits

über 44 Regime. Der exponentielle Slow-Start Fit ergibt $R^2 = 0,901$ mit einer Verdopplungszeit von $T_2 \approx 323$ ms.

Für TCP CUBIC ergibt der kubische Fit $\bar{R}^2 = 0,962$ über 30 Regime. Das Wachstum folgt der kubischen Funktion $W(t) = C \cdot (t - K)^3 + W_{\max}$ (vgl. CUBIC-RFC [5]). Die Slow-Start Phase wird durch den ersten Verlust abgebrochen ($R^2 = 0,871$, Verdopplungszeit $T_2 \approx 759$ ms).

Die Reaktion auf Verluste unterscheidet sich im Reduktionsfaktor: Reno halbiert das cwnd ($\beta = 0,5$), CUBIC reduziert auf 70 % ($\beta = 0,7$). Dies wird durch die gemessenen Daten ebenso widergespiegelt.

6.4.2 Performance, Robustheit und Stabilität

Im Durchsatz sind beide Algorithmen nahezu gleichwertig (Reno: 968 kbit/s, CUBIC: 956 kbit/s). Dies ist auf das niedrige BDP ($\text{BDP} \approx 1 \text{ Mbit/s} \times 80 \text{ ms} \approx 10 \text{ kB}$) zurückzuführen: beide Algorithmen füllen die verfügbare Kapazität, bevor CUBICs aggressiveres Wachstum zum Tragen kommen kann.

TCP CUBIC zeigt eine leicht niedrigere Verlustrate (2,9 % vs. 3,4 %) und weniger Retransmissionen (71 vs. 81). Der höhere Reduktionsfaktor ($\beta = 0,7$) bewirkt, dass CUBIC nach einem Verlust von einem höheren cwnd aus weiterwächst. Der mittlere Outstanding Wert ist dennoch nahezu identisch (Reno: 12.232 Bytes, CUBIC: 11.980 Bytes laut `avg owin` in `{reno,cubic}-summary.txt`).

Hinsichtlich der Latenz ist CUBIC stabiler. Die maximale RTT beträgt 340 ms gegenüber 437 ms bei Reno. Dass der mittlere Outstanding Wert bei CUBIC unter dem von Reno liegt, widerspricht der Erwartung eines höheren cwnd-Durchschnitts durch $\beta = 0,7$, sofern $\text{Outstanding} \approx \text{cwnd}$ angenommen wird. Die niedrigere maximale RTT bei CUBIC fällt hingegen erwartbar aus: Das Wachstum nahe $W_{\max}$ könnte weniger Warteschlangenlatenz erzeugen. Eine mögliche Erklärung für beide Beobachtungen ist, dass die Outstanding-Metrik das tatsächliche cwnd nur ungenau approximiert. Widersprüchlich ist diese Metrik auch an anderer Stelle: Unter der Annahme $\text{Outstanding} \approx \text{cwnd}$ sollten `max owin` und `max snd_cwnd` ähnlich ausfallen, doch `max owin` liegt bei 70.953 Bytes (Reno) bzw. 69.505 Bytes (CUBIC) (vgl. Abschnitt 6.3).

6.4.3 Praktischer Einsatz

Die geringe Differenzierung der Algorithmen ist eine direkte Folge des niedrigen BDPs beziehungsweise der begrenzten zeitlichen Beobachtung. In Netzwerken mit hoher Bandbreite und langer Verzögerung (z. B. transatlantische Verbindungen) wächst cwnd bei CUBIC deutlich schneller als bei Reno und ist daher geeigneter (vgl. CUBIC-RFC [5]).

TCP CUBIC ist in Linux der Standard. Die Messergebnisse bestätigen, dass CUBIC unter den

gegebenen Bedingungen mindestens gleichwertig zu Reno agiert und bei Latenz und Verlustrate leichte Vorteile zeigt.

6.4.4 Grenzen des Labs

Das Experiment unterliegt einigen Einschränkungen, die bei der Interpretation berücksichtigt werden müssen:

- Die künstlich erzeugte Paketverlustrate von 3 % liegt über realistischen Werten.
- Die Bandbreite von 1 Mbit/s führt zu einem niedrigen BDP.
- Die kurze Messdauer von 30 Sekunden begrenzt die Aussagekraft.
- Die Linux-Implementierungen können vom RFC abweichen.
- Das Einstellen der Algorithmen könnte im Experiment unvollständig gewesen sein.
- Das tatsächliche Congestion Window ist nicht direkt beobachtbar ohne Zugriff auf den Kernel. Die Analyse basiert auf der Outstanding Data (unbestätigte Daten) aus `tcptrace`, die das `cwnd` lediglich approximiert. Bei Verlusten oder verzögerten ACKs kann die Outstanding Data vom realen `cwnd` abweichen.

Literatur

- [1] Energy Sciences Network (ESnet). iperf3 documentation, 2026. Accessed: 2026-07-11. URL: <https://software.es.net/iperf/>.
- [2] Andrei Gurtov, Tom Henderson, Sally Floyd, and Yoshifumi Nishida. The NewReno Modification to TCP's Fast Recovery Algorithm. RFC 6582, April 2012. URL: <https://www.rfc-editor.org/info/rfc6582>, doi:10.17487/RFC6582.
- [3] Shawn Ostermann. tcptrace – a tcp dump file analysis tool, 2004. Accessed: 2026-07-27. URL: <https://linux.die.net/man/1/tcptrace>.
- [4] Wireshark Foundation. Wireshark, 2026. Accessed: 2026-07-11. URL: <https://www.wireshark.org/>.
- [5] Lisong Xu, Sangtae Ha, Injong Rhee, Vidhi Goel, and Lars Eggert. CUBIC for Fast and Long-Distance Networks. RFC 9438, August 2023. URL: <https://www.rfc-editor.org/info/rfc9438>, doi:10.17487/RFC9438.